

GitHub Copilot Tour

Your AI pair programmer, built into VS Code

What is a Copilot?

A **Copilot** is an AI assistant that works *alongside* you – it augments you, it doesn't replace you.

Pilot

You make the decisions

You own the outcome

You set the direction

Copilot

AI suggests and assists

AI accelerates the work

AI helps navigate

- Keeps you in control
- Handles repetitive or boilerplate tasks
- Still requires your judgement to review and verify

Copilot + You

Two mental models for thinking about Copilot:

Concept

Think of it like

Copilot

Your IDE

Prompt engineering

A programming language

-
- Like your IDE – a tool you get better at over time
 - Like a programming language – how you communicate determines what you get out

Invest time in learning to use it well

Copilot & Your Workflow

AI **augments** your existing skills – it doesn't replace them.

Skill	How Copilot helps
IDE knowledge	Shortcuts maximise Copilot features
CLI fluency	Pairs well with Agent mode
Communication	Prompting skills = writing skills
Previous experience	Your context + AI's breadth

Copilot amplifies what you already bring to the table

GitHub Copilot

GitHub Copilot is Microsoft & GitHub's AI coding assistant, powered by LLMs (OpenAI, Anthropic, Google).

What it can do:

- Complete code as you type (inline suggestions)
- Answer questions about code and concepts in natural language
- Generate, refactor, explain, and test code
- Work autonomously across your entire workspace (Agent mode)
- Integrate with tools and services via MCP

Available in VS Code, JetBrains, Visual Studio, and GitHub.com

Installing GitHub Copilot

Step 1 – Sign in

Ensure your GitHub account has a Copilot licence

Step 2 – Install the extension

Search `GitHub Copilot` in the Extensions panel (`Cmd+Shift+X`)

Step 3 – Authorise

VS Code prompts you to sign in to your GitHub account

Step 4 – Confirm it's working

Copilot icon appears in the status bar (bottom right)

Also install **GitHub Copilot Chat** for the full chat experience

Modes: Ask

Chat with Copilot about code, concepts, and questions.

- Ask in natural language
- Get explanations of code
- Explore unfamiliar codebases
- Learn concepts interactively

Best for: learning, exploring, quick answers

Modes: Plan (*Preview*)

Copilot describes what it *would* do before making any changes. Plan mode is designed for complex tasks involving large-scale changes. If you want to make a change that may impact multiple files, consider using plan mode to safely explore your codebase before committing to making changes.

- Review the plan before any edits happen
- Approve or adjust scope before implementation begins
- See which files will be touched

Best for: large or risky changes

Think before you act – great for unfamiliar codebases

Modes: Agent

Copilot autonomously edits files, runs commands, and iterates. Use the agent to make simple, well-scoped changes, or combine it with plan mode to meet more complex performance needs.

- Works across multiple files simultaneously
- Can run terminal commands
- Iterates until the task is complete
- Review suggestions before accepting

Best for: building features, scaffolding projects

Most powerful mode – **review every change it makes**

Modes Summary

Mode	What it does	Best for
Ask	Answers questions	Learning & exploring
Plan	Describes changes first	Large / risky changes
Agent	Autonomously edits & runs	Building features

Use **Ask** to learn, **Agent** to build, **Plan** to review first

Models & Use Cases

Choose the right model for the job:

Model	Best for
GPT-4.1	General-purpose coding & writing
Claude Haiku or GPT-5 mini	Cheaper cost, providing fast help with simple or repetitive tasks
Claude Sonnet or GPT-5	Deeper reasoning to help debug and solve more complex problems
GPT 5-3 Codex	An advanced coding specific, software engineering model
Claude Opus	Hardest reasoning and complex tasks

Switch models mid-conversation in Copilot Chat

Full model list on GitHub →

Context Window in GitHub Copilot

The **context window** is everything Copilot can "see" at once.

When it fills up, older information is dropped.

Strategies to manage it:

- Start a **new chat** for each new topic
- Use `#file` to include only relevant files
- Keep conversations focused – one task per chat
- Use Agent mode sparingly for long sessions
- Summarise and restart if a chat gets too long

A fresh, focused chat often gets better results than a long, messy one

Chat Features – Slash Commands /

Built-in actions triggered instantly:

Command	What it does
<code>/explain</code>	Explain selected code
<code>/fix</code>	Suggest a fix for an issue
<code>/tests</code>	Generate unit tests
<code>/doc</code>	Add documentation comments
<code>/new</code>	Scaffold a new project
<code>/clear</code>	Clear chat history

Type `/` in the chat input to see all available commands

Chat Features – Chat Participants @

Route your question to a **specialist context**:

Participant	Scope
@workspace	Your entire project
@vscode	VS Code settings & features
@terminal	Terminal commands & output
@github	GitHub.com issues, PRs, repos

Examples:

- @workspace How does auth work in this project?
- @terminal Why did this command fail?
- @vscode How do I enable format on save?

Chat Features – Chat Variables

Attach **specific context** to your message:

Variable	What it attaches
#file	A specific file
#selection	Your current selection
#editor	Active editor contents
#terminalLastCommand	Last terminal command + output
#codebase	Indexed workspace code

Chat Features – Chat Variables

Examples:

Refactor #file:userService.ts to use async/await

Add tests for #file:calculator.ts

Why did #terminalLastCommand fail?

Use # to be precise about what Copilot should look at

Chat Features – Tools

Tools extend what Copilot can access in Agent mode:

- **Built-in tools** – search files, run terminal commands, read/write files
- **MCP tools** – external services (Context7, GitHub, databases, browser)
- **Custom tools** – defined via MCP server configuration

Enable tools via the **tools icon** (paperclip / expand icon) in the chat input bar

More tools = more capability, but also more responsibility to review

Summary

Feature	Purpose
Ask mode	Learn and explore
Agent mode	Build and automate
Plan mode	Review before changes
/ commands	Quick built-in actions
@ participants	Specialist context routing
# variables	Attach precise context
Tools	Extend Copilot's reach

The more precise your context, the better Copilot's output

Questions?

Resources:

- GitHub Copilot Docs
- VS Code Copilot Overview
- Available AI Models